

Avaliação Final

Big Data para Negócios

TechPay: arquitetura, pipeline, análise e painel de risco de fraude

Aluno: Vitor Dantas

Curso: Big Data para Negócios — UFC / MPCE

Professor responsável: Luiz Alexandre Moreira Barros

Formato: Individual — relatório, repositório de código e dashboard

Ambiente: Rota sem admin (DuckDB, PySpark local e Plotly, em WSL2/Ubuntu)

Data de entrega: ____/____/____

Link do repositório: _____

Sumário

Apresentação do trabalho.....	3
Etapa 1 — Arquitetura de Big Data.....	4
Etapa 2 — Execução do pipeline.....	7
Etapa 3 — Análise e Dashboard.....	11
Relatório de achados.....	11
O dashboard.....	15
Ideia do BI.....	17
Bônus — Modelo preditivo de fraude.....	19
Anexo — Estrutura do repositório e reprodução.....	22

Apresentação do trabalho

Este relatório reúne as três etapas da Atividade Final e o bônus de modelo preditivo, aplicados ao cenário da fintech fictícia **TechPay**, sobre a base `avaliacao_transactions.csv` (30.000 transações de 2025, gerada pelo script do professor com semente fixa). Conforme a regra da avaliação, o texto explica em prosa o que foi feito e por quê; o código está no repositório Git, na pasta de cada etapa.

Ambiente utilizado

Foi adotada a **rota sem admin** em todo o trabalho, a mesma usada nos Labs 1 a 12: WSL2 com Ubuntu no Windows, sem cluster Hadoop administrado. O armazenamento é feito em pastas locais com Parquet particionado por ano e mês, a transformação em DuckDB (no lugar do Hive), a exploração e o modelo em PySpark local, e o painel em Plotly, gerado como um arquivo HTML que abre sem servidor (no lugar do Metabase, que exige Docker com privilégio de administrador). A lógica de cada camada é a mesma que seria executada no cluster.

Como ler este relatório

Cada etapa segue a ordem do enunciado. Os números citados vêm dos logs e tabelas gravados pelos próprios scripts, e o dashboard lê as mesmas tabelas Gold usadas nas análises, por isso os valores coincidem em todo o documento. A execução completa pode ser reproduzida com um único comando, descrito no anexo.

Etapa	Peso	Entregável	Onde está no repositório
1 · Arquitetura	15%	Diagrama e justificativa	<code>avaliacao_final/etapa1_arquitetura/</code>
2 · Pipeline	30%	Código e explicação das 6 etapas	<code>avaliacao_final/etapa2_bigdata/</code>
3 · Análise e BI	25%	Análises, dashboard e ideia do BI	<code>avaliacao_final/etapa3_analise/</code>
Bônus	extra	Regressão logística interpretada	<code>avaliacao_final/etapa4_ml/</code>

Etapa 1 — Arquitetura de Big Data

TechPay — arquitetura de dados para risco de fraude

Batch diário (D-1), arquitetura Medallion
Rota A: cluster Hadoop | Rota B (executada): DuckDB + PySpark local + Plotly

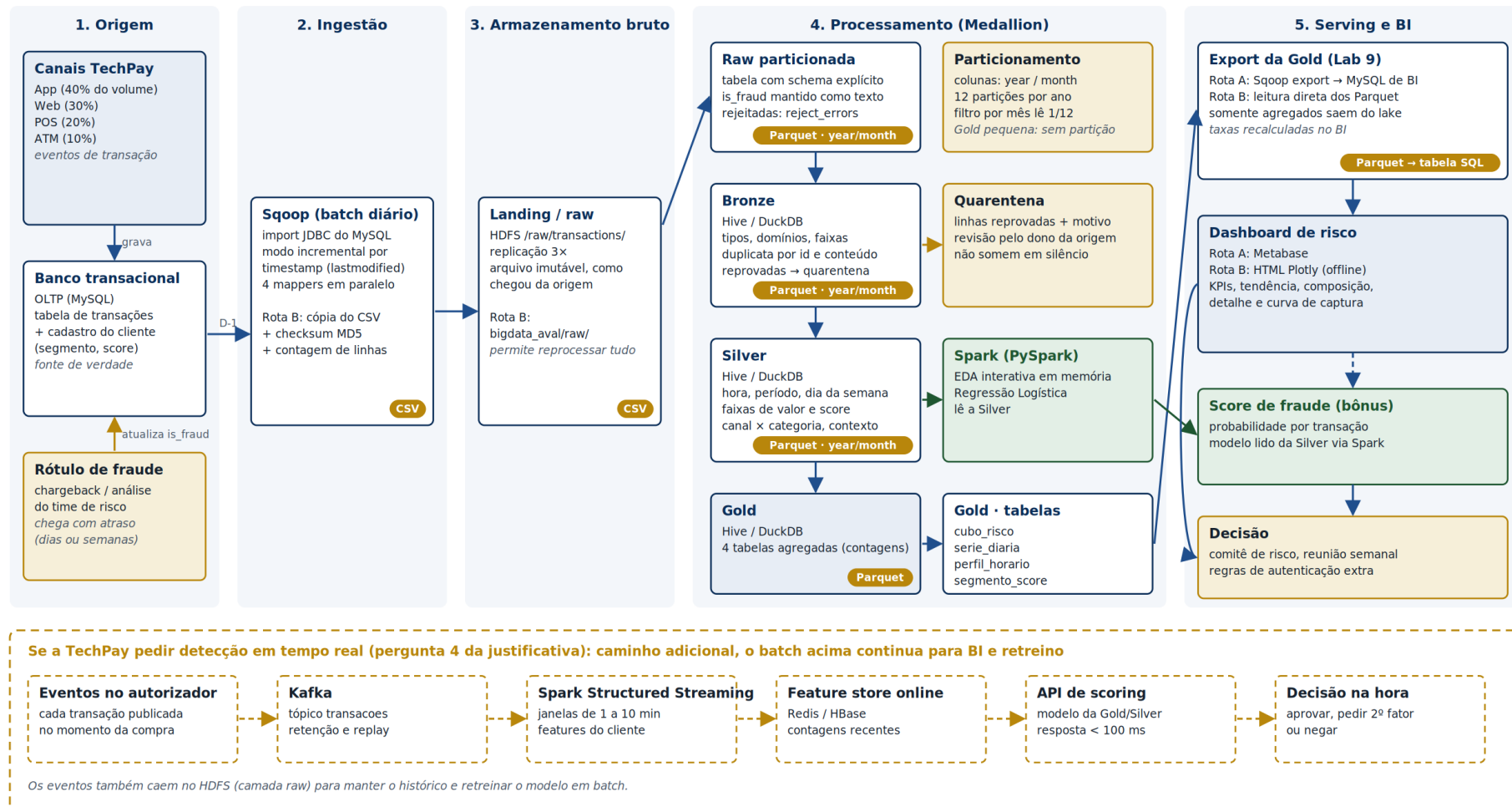


Figura 1 — Arquitetura de dados da TechPay: fluxo da origem à decisão, com ferramenta e formato por camada e o caminho adicional para tempo real

Diagrama: `diagrama.png` (fonte editável em `diagrama.svg`). Cada caixa indica a ferramenta de cluster (Rota A) e a equivalente usada na execução (Rota B: DuckDB, PySpark local e Plotly, no WSL2/Ubuntu, a mesma rota dos Labs 1 a 12).

Fluxo e formatos

As transações nascem nos canais da TechPay (app, web, POS, ATM) e são gravadas no banco transacional, que também guarda segmento e credit score do cliente. O rótulo de fraude chega depois, quando o time de risco conclui a análise, por isso o desenho o trata como atualização tardia.

Uma vez por dia, a ingestão copia as transações do dia anterior para a camada **raw**, em **CSV**, exatamente como chegaram. Isso permite reprocessar tudo se uma regra de limpeza estiver errada. A partir daí, tudo é **Parquet**: o formato é colunar, comprimido (2.678 KB viraram 743 KB) e guarda o tipo de cada coluna, evitando o problema dos Labs 2, 4 e 5 em que o `is_fraud` em texto era convertido errado.

A cadeia Medallion é: raw particionada (schema explícito) → Bronze (limpeza e quarentena) → Silver (colunas de análise) → Gold (4 tabelas de contagens, sem partição, porque a maior tem 3.235 linhas). Só a Gold vai para o BI; o Spark lê a Silver para exploração e para o modelo. O dashboard alimenta o comitê semanal de risco.

1. Ferramenta de ingestão e alternativa

A origem é um banco relacional e a necessidade é analítica, com visão do dia anterior. O **Sqoop** resolve com poucos parâmetros: lê via JDBC, divide a leitura em mappers paralelos, grava no HDFS e, no modo incremental, traz só o que mudou desde a última carga. Na Rota B, o equivalente foi copiar o arquivo e conferir assinatura MD5 e contagem de linhas.

A alternativa seria **Kafka** (ou Flume/NiFi), recebendo cada transação no momento em que ocorre. Não usei porque o pedido é batch: streaming traria brokers, tópicos e consumidores para operar, sem ganho numa análise olhada semanalmente. Ressalva: o Sqoop foi descontinuado pela Apache em 2021; em produção, eu faria a mesma leitura incremental com o próprio Spark.

2. Estratégia de particionamento

Partições por **ano e mês** da transação. A consulta mais frequente da área de risco é recortada no tempo ("como fechou a fraude do mês e como compara com o anterior?") e, com essa partição, abre 2 das 12 pastas. A execução confirmou: uma consulta filtrada em março leu 1 de 12 arquivos.

Partição por **dia** foi descartada: com 30 mil transações no ano, seriam 365 pastas com cerca de 80 linhas cada, o clássico problema de arquivos pequenos. Partição por **canal** também: quase nenhuma consulta filtra só o canal sem recortar o período, e esse filtro acontece no dashboard, sobre a Gold. Com 100 vezes mais volume, cada mês teria cerca de 250 mil linhas, ainda saudável; acima disso, o próximo passo seria particionar por dia.

3. O que quebra primeiro com 100 vezes o volume

1. **Carga na origem.** Ler a tabela inteira todo dia pesaria no banco que atende os clientes. Mitigação: importação incremental, leitura numa réplica e janela noturna.
2. **Máquina única da Rota B.** O DuckDB está limitado a um computador e não tem tolerância a falha. O mesmo pipeline passaria a rodar em Spark ou Hive no cluster.
3. **Reprocessamento completo.** Recriar todas as camadas a cada carga deixa de ser viável; o certo é reprocessar só a partição do mês com dados novos.
4. **Arquivos pequenos e desbalanceamento.** Cargas diárias multiplicam arquivos por partição, e o app concentra 40% do volume. Mitigação: compactação periódica e atenção à distribuição nas agregações.
5. **Metastore e NameNode.** O metastore Derby, que deu problema no curso, não aceita vários usuários; em produção seria MySQL ou PostgreSQL, com NameNode em alta disponibilidade.
6. **Rótulo atrasado.** Fraudes confirmadas semanas depois exigem reprocessar meses anteriores, o que só é possível porque a raw é imutável e particionada.

O que não piora: a tabela principal da Gold cresce com o número de combinações de mês, canal, categoria, segmento e período, não com o número de transações, então o dashboard continua leve.

4. Detecção de fraude em tempo real

A decisão passaria a acontecer enquanto o cliente espera a aprovação. A faixa tracejada do diagrama mostra o caminho adicional: o autorizador publica cada transação no **Kafka**; o **Spark Structured Streaming** calcula características recentes do cliente em janelas de minutos (por exemplo, compras de viagem pelo app na última hora); elas ficam num armazenamento de baixa latência (**Redis** ou **HBase**); uma **API de scoring** aplica o modelo treinado no batch e responde em menos de 100 ms: aprovar, pedir segundo fator ou negar.

O batch continua existindo: os eventos também caem na raw, de onde saem o histórico, o BI e o retreino. As mudanças principais são a ingestão por eventos, uma camada de serving de baixa latência, o monitoramento contínuo do modelo e o tempo de resposta tratado como requisito de negócio.

Etapa 2 — Execução do pipeline

Rota declarada: Rota B (sem admin), a mesma dos Labs 1 a 12. O pipeline roda em DuckDB, gravando Parquet particionado em pasta local, no WSL2/Ubuntu. Não houve tentativa de cluster nesta etapa. A ferramenta muda, mas a lógica de cada camada é a mesma que o Hive executaria.

Arquivos desta pasta:

- `generate_avaliacao_dataset.py`: gerador do professor, sem alterações. Produz a base com 30.000 linhas.
- `pipeline.py`: o pipeline completo, das 6 etapas.
- `teste_regras_bronze.py`: teste das regras de limpeza com linhas sujas injetadas numa cópia da base.
- `evidencias/execucao_log.txt`: saída real da execução (schema, contagens, partições, plano de consulta, conferências).
- `evidencias/teste_regras_bronze_log.txt`: resultado do teste de regras.

Funil de linhas

Camada	Linhas	Observação
Arquivo CSV de origem	30.000	sem contar o cabeçalho
Raw	30.000	0 linhas rejeitadas pelo schema
Raw particionada	30.000	12 partições (jan a dez/2025), de 2.317 a 2.625 linhas cada
Bronze	30.000	0 linhas em quarentena
Silver	30.000	nenhuma perda ou duplicação
Gold	4 tabelas	a soma das transações em cada tabela confere com 30.000

As 6 etapas

1. Ingestão

O que fez. Copiou o arquivo gerado para a pasta de entrada da camada raw, sem mexer em nenhum byte.

Decisão. Não basta copiar: o pipeline calcula a assinatura MD5 do arquivo de origem e da cópia e conta as linhas. Assim fica provado que o que entrou no ambiente é idêntico ao que saiu da origem. As duas assinaturas bateram e a contagem deu 30.000 linhas.

Problema real. O gerador do professor grava o arquivo numa pasta temporária do sistema. O pipeline recebe o caminho do arquivo como parâmetro e para com uma mensagem clara se o arquivo não existir, em vez de seguir com uma base vazia.

2. Tabela raw

O que fez. Criou a tabela raw declarando o tipo de cada uma das 12 colunas, sem deixar a ferramenta adivinhar.

Decisão. A coluna `is_fraud` ficou deliberadamente como texto nesta camada. Nos Labs 2, 4 e 5 vimos que converter "True"/"False" direto para booleano pode zerar todas as fraudes sem dar erro, então a conversão ficou para a Bronze, onde é feita de forma explícita. Também ativei o registro de linhas rejeitadas: uma linha que não respeita o schema (por exemplo, texto no campo de valor) vai para uma tabela de rejeitos, em vez de derrubar a carga inteira.

O que foi encontrado. 0 rejeições. O rótulo chegou como 29.249 "False" e 751 "True".

3. Particionamento

O que fez. Aplicou a estratégia da Etapa 1: gravou a raw em Parquet, separada em pastas por ano e mês. Resultaram 12 pastas.

Decisão. Particionar já nesta camada para que a Bronze e qualquer consulta posterior aproveitem o corte por período. A evidência está no log: uma consulta filtrada em março mostrou no plano de execução que só 1 dos 12 arquivos foi lido. Os volumes mensais ficaram equilibrados (fevereiro é o menor, com 2.317 linhas; janeiro o maior, com 2.625), então não há partição desproporcional.

Ganho de espaço. A troca de CSV por Parquet reduziu o armazenamento de 2.678 KB para 743 KB.

4. Bronze

O que fez. Limpeza sem regra de negócio:

- padronizou texto (espaços nas pontas; minúsculas em canal, categoria, tipo e status);
- converteu o rótulo de fraude aceitando só valores conhecidos;
- aplicou 14 regras de reprovação.

As regras cobrem identificador nulo, duplicata por `transaction_id`, duplicata por conteúdo (a mesma transação com outro id), valor zero, negativo ou acima de R\$ 40.000, data fora de 2025, valores fora do domínio esperado em tipo, canal, categoria, status e segmento, risk score fora de 0 a 100, credit score fora de 300 a 900, cliente fora da faixa 1 a 8.000, e rótulo inválido. Na duplicata, fica a primeira ocorrência.

Decisão 1: quarentena. Linha reprovada não é apagada em silêncio. Ela vai para uma tabela de quarentena com o motivo, para que alguém possa corrigir na origem.

Decisão 2: sinalizar sem descartar. Alguns casos são válidos, mas o analista precisa saber deles. Nesses, a linha fica e recebe uma sinalização.

O que foi encontrado. A base oficial é limpa por construção: **nenhuma das 14 regras reprovou linha alguma.** Para mostrar que a Bronze não é uma cópia, rodei o `teste_regras_bronze.py`. Ele injeta 12 linhas sujas numa cópia da base e roda o mesmo pipeline em outra pasta. Resultado:

- 10 linhas foram para a quarentena, cada uma com o motivo certo: a cópia de uma transação existente (a original permaneceu), a duplicata com outro id e as 8 linhas com valor, data, score, canal, rótulo ou cliente inválido;
- a linha com texto no campo de valor foi barrada já na raw, pelo schema;
- a linha com canal "APP" e categoria "Viagem" foi corrigida para "app" e "viagem" e aceita;
- a Bronze do teste terminou com 30.001 linhas, exatamente o esperado.

A base oficial também tem problemas reais que não são motivo de descarte:

- **Valores no piso.** 573 transações têm valor exatamente R\$ 5,00. O gerador corta tudo que ficaria abaixo disso, então esses valores não são reais. Ficaram marcados.
- **Credit score nos limites.** 1.222 transações têm credit score exatamente 300 ou 900, pelo mesmo motivo. Também marcadas.
- **Rótulo × status.** 100% das 751 fraudes estão com status "declined", e 2.301 transações legítimas também foram recusadas. Nenhuma fraude aparece como aprovada. Consequência: o status é praticamente uma consequência do rótulo, e usá-lo num modelo seria vazamento de informação. Ele ficou fora do modelo do bônus.
- **Segmento inconsistente por cliente.** Dos 7.803 clientes, 5.887 aparecem com mais de um segmento em transações diferentes. Não há como saber qual é o correto, então tratei o segmento como a classificação do cliente no momento da transação, e não como atributo fixo. Por isso não criei uma tabela de clientes única a partir desta base.
- **Horário só em hora cheia.** Todos os minutos e segundos são 00. Toda análise temporal foi feita no máximo por hora.

5. Silver

O que fez. Acrescentou as colunas de contexto que as etapas seguintes usam, e só elas:

- data e hora;
- dia da semana (segunda = 1, com o nome abreviado);
- período do dia;
- faixa de credit score (300–499, 500–699, 700–900);
- faixa do risk score atual (abaixo de 60, 60–79, 80 ou mais);
- indicação de recusa (status "declined").

A Silver ficou com 22 colunas: as 12 originais, ano e mês (que vêm das pastas da partição) e as 8 derivadas acima.

Decisão sobre o que entra. A regra foi manter na Silver apenas o que alguma tabela Gold, análise ou o modelo realmente utiliza. Colunas cogitadas e deixadas de fora:

- faixa de valor e dia do mês: o valor não separa fraude, e o dia do mês não entrou em nenhuma análise;
- indicação de fim de semana: a análise por dia da semana já cobre;
- combinação canal + categoria: a Gold guarda as duas dimensões separadas, e o cruzamento é feito na agregação;
- colunas de histórico do cliente (quantidade de transações no ano, valor em relação à média dele): seriam calculadas com o ano inteiro, o que usaria informação do futuro em relação a cada transação.

As sinalizações de qualidade (valor no piso, score no limite) ficaram só na Bronze, onde são contadas e diagnosticadas.

Decisão sobre o período do dia. Usei uma convenção fixa, definida antes de olhar a fraude: madrugada de 0h a 5h59, manhã de 6h a 11h59, tarde de 12h a 17h59 e noite de 18h a 23h59. Assim o corte não é ajustado aos dados. A análise hora a hora da Etapa 3 mostra que isso tem um custo pequeno: as horas de 0h a 4h ficam todas acima de 3% de fraude, e às 5h a taxa já é 2,30%, o que dilui um pouco a taxa da madrugada.

Problema real resolvido. No curso, a Silver fazia um JOIN com a tabela de clientes para trazer segmento e credit score, e perdia cerca de 46 transações de clientes inexistentes. Nesta base, esses campos já vêm na própria transação, então não houve JOIN nem perda: a Silver tem as mesmas 30.000 linhas da Bronze. Com o segmento inconsistente por cliente (item 4), fazer esse JOIN teria sido um erro.

6. Gold

O que fez. Criou 4 tabelas agregadas, todas com as mesmas medidas: transações, fraudes, recusas, recusas de clientes legítimos, valor total e valor fraudado.

- `gold_cubo_risco` (3.235 linhas): uma linha por mês × canal × categoria × segmento × período do dia.
- `gold_serie_diaria` (1.460 linhas): uma linha por dia × canal.
- `gold_perfil_horario` (168 linhas): hora × dia da semana.
- `gold_segmento_score` (27 linhas): segmento × faixa de credit score × faixa do risk score.

Decisão. A Gold guarda **contagens e somas, não taxas**. Uma taxa de fraude não pode ser somada nem tirada a média entre grupos de tamanhos diferentes. Guardando fraudes e transações, o dashboard recalcula a taxa correta para qualquer combinação de filtros.

Conferência. Em cada tabela, a soma das transações deu 30.000. Por segmento, a Gold reproduz os números do gerador do professor: High-Risk 9,67%, Standard 2,93%, Premium 0,95%.

Respostas às perguntas do enunciado

1. Quantas linhas sobreviveram da Bronze em diante? Alguma foi descartada?

As 30.000 sobreviveram em todas as camadas; nenhuma foi descartada. Não é falta de limpeza:

- as 14 regras foram aplicadas e contadas uma a uma, e todas deram zero nesta base;

- o teste com linhas sujas prova que elas funcionam;
- os problemas que existem (valores no piso, scores nos limites, segmento inconsistente) foram sinalizados ou tratados na interpretação, porque descartá-los apagaria transações verdadeiras.

2. A Silver usa `channel` e `merchant_category`? Por quê?

Sim, as duas, e também a combinação delas. O gerador foi construído para que a fraude dependa desses campos, e a exploração confirmou:

- **canal:** o app tem 3,79% de fraude, contra 1,42% a 1,80% nos outros canais;
- **categoria:** viagem tem 5,32%, contra 1,96% a 2,47% nas demais;
- **combinação:** app + viagem chega a 8,00%.

Sem esses campos, a Silver esconderia justamente onde a fraude se concentra, e a análise ficaria presa ao segmento, como no Lab 12. As duas dimensões seguem juntas até a Gold (no cubo de risco), porque o efeito conjunto é maior que cada um separado, e é lá que o cruzamento é calculado.

3. Que agregações foram para a Gold e o que elas respondem?

- **Cubo de risco:** alimenta os KPIs, os gráficos de composição por canal, categoria e segmento, a tabela de detalhe e a curva de captura do dashboard. Responde às análises de segmento, canal, categoria e período.
- **Série diária por canal:** alimenta a média móvel de 7 dias, que mostra se a fraude está subindo ou descendo.
- **Perfil hora × dia da semana:** responde à análise temporal fina (qual hora e qual dia concentram fraude).
- **Segmento × faixa de score:** responde se o credit score e o risk score atual separam fraude, que é a base da pergunta de negócio própria da Etapa 3.

Etapa 3 — Análise e Dashboard

Relatório de achados

Como os números foram produzidos: `analises.py` lê a Gold (e a Silver quando precisa do nível da transação). Cada tabela citada está em `resultados/` e todas estão reunidas em `resultados/resumo_analises.md`. O dashboard (`dashboard.html`, gerado por `dashboard.py`) lê as mesmas tabelas Gold, por isso os números batem.

Convenções:

- **Taxa de fraude** = fraudes ÷ transações do grupo.
- **Lift** = taxa do grupo ÷ taxa geral; 1,00 significa igual à média.
- A taxa geral da base é **2,50%**: 751 fraudes em 30.000 transações, somando R\$ 119.368 fraudados de R\$ 5.207.844 transacionados (2,29% do valor).

Foram feitas as cinco análises da lista. Cada uma segue o framework da aula: Finding → Insight → Ação.

Análise 1 — Segmentação por segmento e score

Finding.

Segmento	% do volume	% das fraudes	Taxa
High-Risk	9,7%	37,5%	9,67%
Standard	35,5%	41,5%	2,93%
Premium	54,8%	20,9%	0,95%

O High-Risk tem 10 vezes a taxa do Premium. O credit score não acompanha essa diferença:

- o score médio é praticamente igual nos três segmentos (High-Risk 635, Standard 640, Premium 638);
- por faixa de score, a taxa não cai quando o score sobe: 1,97% (300–499), 2,47% (500–699), 2,80% (700–900);
- isolado, o credit score separa fraude quase como um sorteio (AUC 0,529, onde 0,50 é aleatório).

Insight. Nesta base, o segmento carrega o sinal de fraude e o credit score não. Faz sentido: credit score mede chance de inadimplência, não de a transação ser fraudulenta. A pequena diferença entre faixas vai no sentido contrário ao intuitivo e não tem uso prático. O segmento também se combina com o canal: High-Risk no app chega a **13,78%** de fraude, com 3,9% do volume e 21,6% de todas as fraudes.

Ação. Não usar credit score como filtro de fraude. Usar o segmento como primeira camada das regras, começando pelo High-Risk no app, que tem a maior taxa entre os 12 cruzamentos de segmento e canal.

Análise 2 — Risco por canal

Finding.

Canal	Taxa	Lift	% do volume	% das fraudes
App	3,79%	1,51	40,1%	60,6%
ATM	1,80%	0,72	10,0%	7,2%
Web	1,74%	0,70	30,0%	20,9%
POS	1,42%	0,57	19,9%	11,3%

Em valor, o app responde por R\$ 63.910, ou 53,5% de todo o valor fraudado.

Insight. O app não é arriscado só porque atende mais clientes High-Risk. Dentro de **cada** segmento, ele é o canal com maior taxa:

- Premium: 1,50% no app, contra no máximo 0,68% nos outros canais;
- Standard: 4,54% contra no máximo 2,26%;
- High-Risk: 13,78% contra no máximo 7,44%.

O efeito é do canal. O POS, onde o cartão está presente fisicamente, é o mais seguro.

Ação. Concentrar a autenticação extra no app e deixar o POS com o fluxo atual. Para os segmentos Standard e High-Risk no app, que somam 18,2% do volume e 47,5% das fraudes, vale avaliar um segundo fator em transações fora do padrão do cliente.

Análise 3 — Risco por categoria de estabelecimento

Finding. Viagem tem 5,32% de fraude (lift 2,12). Com 10,0% do volume, concentra 21,3% das fraudes e R\$ 25.966 fraudados. As outras cinco categorias ficam entre 1,96% (eletrônico) e 2,47% (saúde), todas perto ou abaixo da média. Viagem é a categoria mais arriscada em todos os canais:

Canal	Taxa em viagem
App	8,00%
POS	3,76%
ATM	3,72%
Web	3,14%

Insight. O risco de viagem se soma ao do app. A combinação **app + viagem** tem 8,00% de fraude, acima de cada efeito isolado (3,79% do app e 5,32% de viagem). Com apenas 4,1% do volume (1.237 transações), ela concentra 13,2% das fraudes. Eletrônico, por sua vez, tem a menor taxa da base (1,96%).

Ação. Exigir confirmação adicional em compras de viagem pelo app. É a regra com menor atrito entre as que usam só canal e categoria.

Análise 4 — Padrão temporal

Finding.

- **Período do dia.** A madrugada (0h a 5h59, convenção fixa definida na Silver) tem 3,27% de fraude (lift 1,31) e concentra 32,9% das fraudes com 25,2% do volume. Manhã, tarde e noite ficam entre 2,17% e 2,29%.
- **Hora a hora.** As horas de 0h a 4h estão todas acima de 3%, com pico à 1h (4,46%). Às 5h a taxa já é 2,30%.
- **Dia da semana.** Não há padrão: todos os dias ficam entre 2,32% (segunda) e 2,74% (domingo).
- **Mês.** Também não: a taxa oscila entre 2,08% (setembro) e 2,80% (dezembro), sem tendência de alta ou de baixa ao longo do ano.
- **Combinação.** App + viagem de madrugada chega a 9,27%, com 313 transações e 29 fraudes.

Insight. O sinal temporal está na hora, não no calendário. Isso é diferente da base usada no curso, onde a fraude acompanhava o fim do mês, e mostra por que a exploração precisou ser refeita em vez de reaproveitar as conclusões do Lab 8. O risco concentra-se entre 0h e 4h; a convenção de madrugada até 5h59 inclui uma hora já próxima da média, o que dilui um pouco a taxa, mas evita um corte ajustado aos próprios dados. Como a série mensal é estável, uma alta futura na tendência do dashboard será sinal de mudança real, e não sazonalidade conhecida.

Ação. Incluir a madrugada como critério nas regras do app e não criar regras por dia da semana ou por mês, que não têm base nos dados.

Análise 5 — Pergunta própria: o motor de risco atual aponta para as transações certas?

Pergunta. A TechPay já tem um `risk_score` (0 a 100) em cada transação. Se a empresa aceitar exigir autenticação extra em cerca de 20% das transações, qual critério alcança mais fraudes: o score atual ou regras construídas a partir das análises 1 a 4?

Finding. O score atual quase não separa fraude:

- por faixa, a taxa é 2,37% abaixo de 60, 2,08% entre 60 e 79 e 3,32% a partir de 80;
- a faixa intermediária tem até menos fraude que a mais baixa;
- isolado, o score tem AUC de 0,535.

Simulação das regras sobre as 30.000 transações:

Regra	% do volume alertado	% das fraudes alcançadas	Fraude dentro do grupo	Transações legítimas afetadas
R1: <code>risk_score</code> ≥ 80 (motor atual)	20,1%	26,6%	3,32%	5.833
R2: app + viagem	4,1%	13,2%	8,00%	1.138
R3: High-Risk no app	3,9%	21,6%	13,78%	1.014
R4: todo High-Risk	9,7%	37,5%	9,67%	2.633
R5: High-Risk ou app + viagem	13,4%	46,5%	8,67%	3.675
R6: High-Risk ou app + viagem ou app na madrugada	21,7%	56,7%	6,55%	6.075

Insight. Com volume alertado parecido (21,7% contra 20,1%), a regra R6 alcança **mais que o dobro** das fraudes do motor atual (426 contra 200). O preço é pequeno: 242 transações legítimas a mais passam pela etapa extra (6.075 contra 5.833), enquanto a taxa de fraude dentro do grupo alertado quase dobra (6,55% contra 3,32%). O motor atual só tem algum efeito acima de 80 pontos e ignora exatamente os campos onde a fraude se concentra: canal, categoria, segmento e hora.

Há um segundo achado, sobre atrito: das 3.052 transações recusadas no ano, **2.301 (75,4%) eram legítimas**. Isso equivale a 7,87% de todas as transações legítimas. O custo de uma regra mal calibrada não é só a fraude que passa, é também o cliente bom que é barrado.

Limitação: as regras foram escolhidas olhando esta mesma base, então os percentuais tendem a ser otimistas. O modelo do bônus (Etapa 4), avaliado em dados que ele não viu no treino, alcançou 55,0% das fraudes nos 20% mais suspeitos, um resultado próximo ao da R6. Isso indica que o ganho não é artefato da escolha das regras.

Ação.

1. Levar ao comitê de risco a comparação R1 × R6.
2. Começar por um piloto de R3 (High-Risk no app), que afeta só 3,9% do volume e já alcança 21,6% das fraudes.
3. Ampliar para R6 depois de medir no piloto quantos clientes legítimos abandonam a transação na etapa extra.
4. Em paralelo, recalibrar o `risk_score` incluindo canal, categoria, segmento e período, que é o que o modelo da Etapa 4 faz.

Onde cada análise aparece no dashboard

Análise	Bloco do dashboard
1. Segmentação	Gráfico "Taxa de fraude por segmento"; filtro de segmento em todos os blocos
2. Canal	Gráfico "Taxa de fraude por canal"; mapa canal × categoria
3. Categoria	Gráfico "Taxa de fraude por categoria"; mapa canal × categoria
4. Temporal	Tendência mensal; média móvel de 7 dias; mapa hora × dia da semana; filtro de período
5. Pergunta própria	Bloco "Onde aplicar autenticação extra": curva de captura com o ponto do motor atual e tabela acumulada; KPI de clientes legítimos recusados

O dashboard

O painel (`dashboard.html`) abre com duplo clique, sem internet. As figuras abaixo mostram a versão entregue, sem filtros aplicados. Todos os blocos respondem aos filtros do topo, exceto a média móvel diária (só canal) e o mapa hora × dia da semana (visão da base inteira).

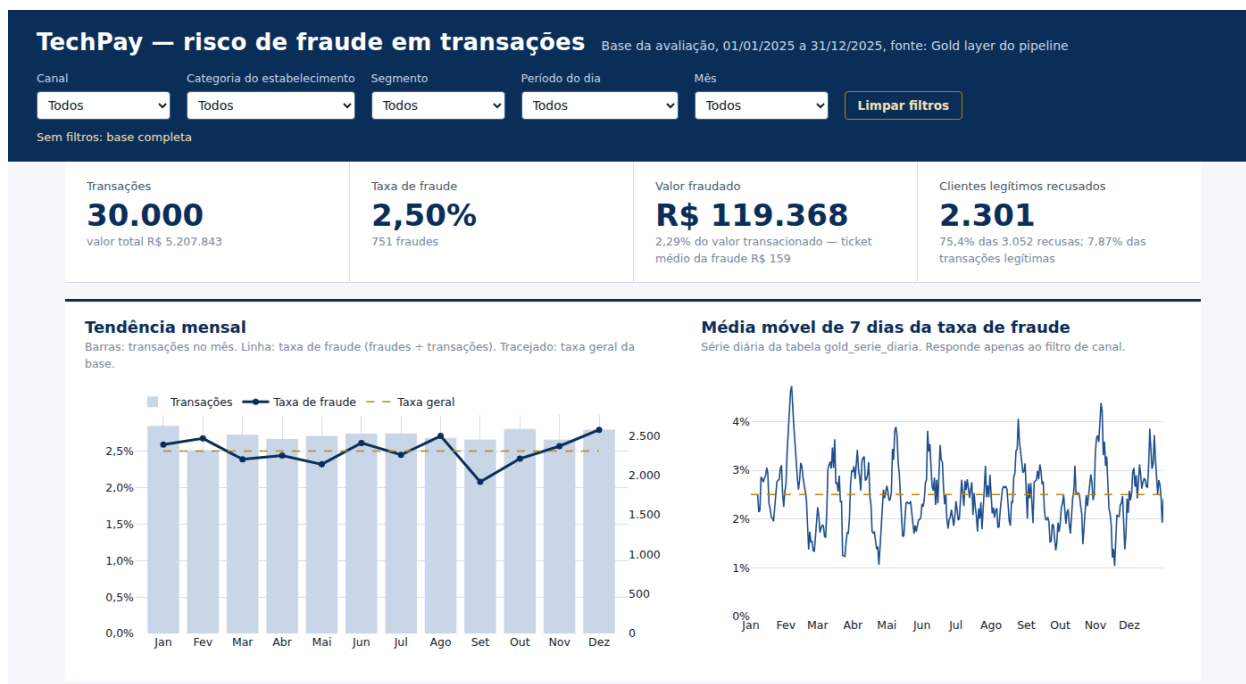


Figura 2 — Filtros, KPIs e bloco de tendência

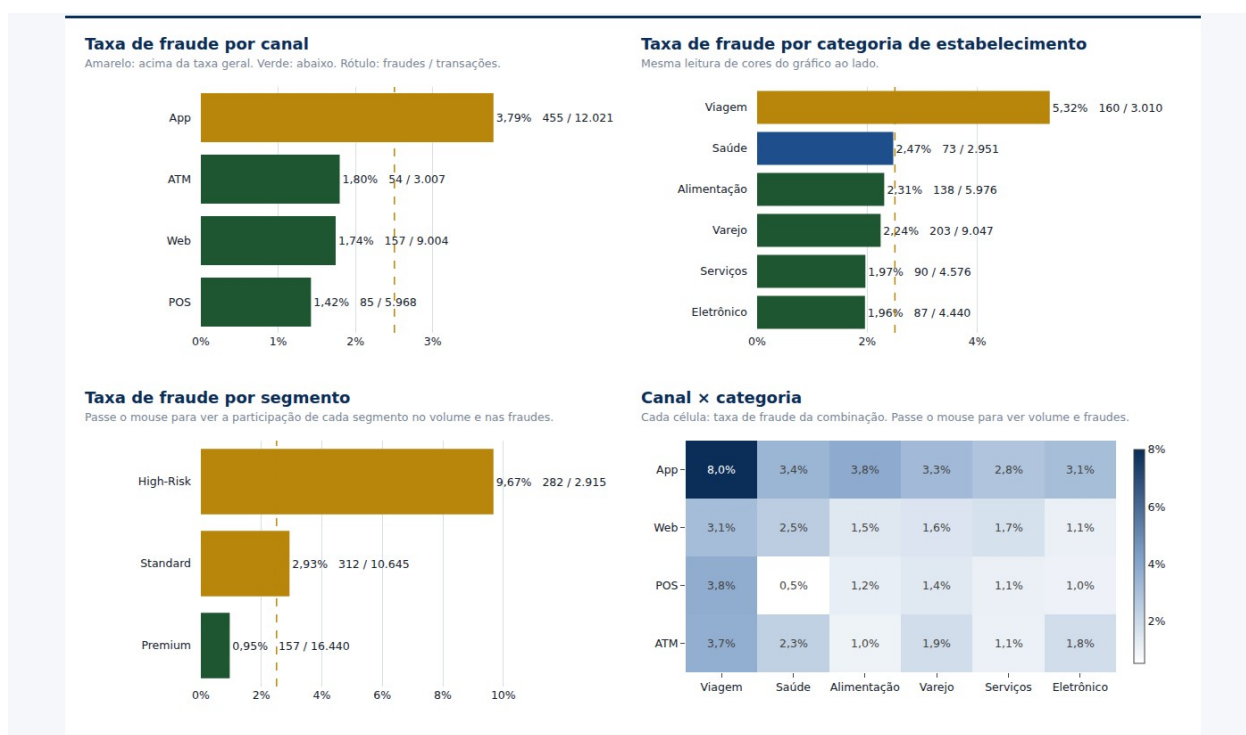
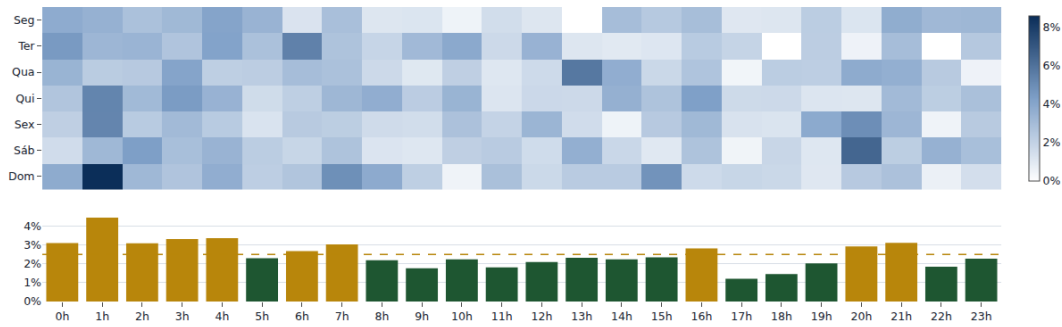


Figura 3 — Bloco de composição: canal, categoria, segmento e mapa canal × categoria

Hora do dia x dia da semana

Taxa de fraude por hora e dia (tabela gold_perfil_horario, visão da base inteira, não responde aos filtros).



Onde aplicar autenticação extra

Combinações de canal, categoria, segmento e período ordenadas da maior para a menor taxa de fraude. Combinações com poucas transações são reunidas num grupo mais amplo ("Demais" indica a dimensão agrupada). A curva mostra quanto das fraudes do recorte seria alcançado ao exigir autenticação extra nas combinações do topo, e quanto do volume seria afetado. Números calculados sobre a própria base de 2025, sem validação fora da amostra; quanto menor o agrupamento mínimo, mais otimista fica a curva.

Volume submetido à autenticação extra: 20% das transações do recorte Reunir combinações com menos de transações

Autenticação extra nos 51 grupos do topo: 6.208 transações (20,7% do recorte) alcançam 474 fraudes (63,1% do recorte). Taxa de fraude dentro do grupo: 7,64%; 5.734 transações legítimas passariam pela etapa extra.



Canal	Categoria	Segmento	Período	x taxa geral	Transações	Fraudes	Taxa %	R\$ fraude	Volume acum.	Fraudes acum.
App	Varejo	High-Risk	Madrugada	10,1x	87	22	25,29%	R\$ 1.224	0,3%	2,9%
App	Viagem	High-Risk	Demais	10,0x	128	32	25,00%	R\$ 5.506	0,7%	7,2%
App	Alimentação	High-Risk	Manhã	6,4x	62	10	16,13%	R\$ 715	0,9%	8,5%
Web	Viagem	High-Risk	Demais	6,0x	80	12	15,00%	R\$ 1.532	1,2%	10,1%
App	Alimentação	High-Risk	Tarde	5,9x	54	8	14,81%	R\$ 850	1,4%	11,2%
App	Viagem	Standard	Madrugada	5,7x	105	15	14,29%	R\$ 1.251	1,7%	13,2%
App	Alimentação	High-Risk	Madrugada	5,6x	78	11	14,10%	R\$ 2.312	2,0%	14,6%
App	Eletrônico	High-Risk	Demais	5,6x	172	24	13,95%	R\$ 4.616	2,6%	17,8%
App	Saúde	High-Risk	Demais	5,1x	110	14	12,73%	R\$ 2.224	2,9%	19,7%
App	Serviços	High-Risk	Demais	5,1x	126	16	12,70%	R\$ 1.289	3,3%	21,8%
POS	Viagem	High-Risk	Demais	5,1x	71	9	12,68%	R\$ 893	3,6%	23,0%
App	Viagem	Standard	Noite	4,2x	113	12	10,62%	R\$ 3.181	4,0%	24,6%
ATM	Demais	High-Risk	Demais	3,8x	136	13	9,56%	R\$ 3.316	4,4%	26,4%

Figura 4 — Padrão hora x dia da semana e bloco de detalhe "Onde aplicar autenticação extra"

Ideia do BI

Arquivo: `dashboard.html`. Abre com duplo clique, funciona sem internet e sem servidor, porque a biblioteca de gráficos e os dados da Gold vão embutidos no arquivo. É gerado por `dashboard.py` a partir de `bigdata_aval/gold/`. A captura `dashboard_print.png` mostra a versão entregue.

Rota declarada: sem admin (Plotly em HTML), no lugar do Metabase, que exige Docker com privilégio de administrador.

O painel como produto

O painel não foi pensado como um conjunto de gráficos sobre fraude. Ele foi pensado para responder a uma pergunta operacional: **em quais transações a TechPay deve pedir uma confirmação extra ao cliente, e quanto isso custa em atrito?** Tudo na tela serve a essa decisão.

Estrutura, de cima para baixo, seguindo a anatomia de 4 blocos da aula:

1. **Filtros** de canal, categoria, segmento, período do dia e mês. Valem para todos os blocos, exceto dois gráficos que indicam isso na própria legenda.
2. **KPIs**. São quatro números:
 - transações;
 - taxa de fraude, comparada com a taxa geral quando há filtro;
 - valor fraudado, com o ticket médio da fraude;
 - clientes legítimos recusados, com o percentual das recusas que eram legítimas.
3. **Tendência**. Taxa de fraude mês a mês sobre o volume, e a média móvel de 7 dias, com a taxa geral como linha de referência.
4. **Composição**. Taxa por canal, por categoria e por segmento, com cores que indicam acima ou abaixo da média; mapa canal × categoria; mapa hora × dia da semana.
5. **Detalhe navegável: "Onde aplicar autenticação extra"**. As combinações de canal, categoria, segmento e período aparecem ordenadas da maior para a menor taxa. Um controle deslizante escolhe quanto do volume passaria pela etapa extra. O painel mostra, para essa escolha:
 - quantas fraudes seriam alcançadas;
 - qual a taxa de fraude dentro do grupo;
 - quantas transações legítimas passariam pela etapa extra;
 - no gráfico, onde está o motor atual ($\text{risk_score} \geq 80$), para comparação.

Combinações com menos de 50 transações (valor ajustável) são reunidas num grupo mais amplo: primeiro juntando os períodos, depois as categorias, depois os canais. Assim o painel não premia grupos pequenos por acaso, e toda linha continua sendo uma regra que dá para aplicar (por exemplo, "App, Viagem, High-Risk, demais períodos").

1. Para quem é o painel?

Para o **comitê semanal de prevenção a fraudes**, conduzido pelo gerente de prevenção a fraudes e com os analistas de risco. O analista usa os filtros e a tabela de detalhe para preparar a reunião; o gerente usa os KPIs, a curva e o texto-resumo para decidir.

Não foi desenhado para a diretoria nem para o atendimento:

- a diretoria precisa de um resumo mensal de perdas, que cabe nos KPIs do topo, mas não de uma ferramenta de calibração de regras;
- o atendimento precisa consultar uma transação específica de um cliente, o que exige dados por transação, e a Gold não guarda isso de propósito.

2. Que decisão ele ajuda a tomar?

Quais combinações de canal, categoria, segmento e horário entram (ou saem) da regra de autenticação extra na próxima semana, e com que abrangência.

Um exemplo de uso com os dados de 2025: com o controle em 20%, os 51 grupos do topo somam 20,7% do volume e alcançam 63,1% das fraudes, enquanto o motor atual, com 20,1% do volume, alcança 26,6%. Esse número é calculado sobre a própria base e tende a ser otimista; o modelo da Etapa 4, avaliado fora da amostra, alcança 55,0% com 20% do volume. Ao mover o controle para 5% ou 10%, o comitê vê na hora quantas fraudes deixa de alcançar e quantos clientes legítimos deixa de incomodar. A escolha é negociada ali, com o número na tela.

Uma segunda decisão, mais lenta: **se o motor de risco atual precisa ser recalibrado**. O ponto verde do motor atual, muito abaixo da curva, é a evidência visual para isso.

3. Por que esses KPIs, e o que foi descartado?

Os que ficaram:

- **Taxa de fraude**, e não a quantidade de fraudes sozinha. A quantidade sobe e desce com o volume; a taxa permite comparar canais e segmentos de tamanhos muito diferentes (o app tem 4 vezes o volume do ATM).
- **Valor fraudado**. É a perda que o comitê precisa justificar, e mostra que a fraude pesa 2,29% do valor transacionado.
- **Clientes legítimos recusados**. É o custo do outro lado da balança. Nesta base, 75,4% das recusas foram de clientes legítimos. Sem esse número, o painel empurraria o comitê a apertar as regras indefinidamente.
- **Transações**. Dá a escala do recorte filtrado e evita conclusões sobre grupos pequenos.

Os que foram descartados, e por quê:

- **Ticket médio geral e faixa de valor**. O valor não separa fraude (AUC 0,496, igual a um sorteio), e o ticket médio da fraude (R\$ 159) é parecido com o geral (R\$ 174). O ticket da fraude aparece só como contexto no KPI de valor.
- **Credit score médio**. Não diferencia fraude nesta base (análise 1). Mostrá-lo induziria a regras erradas.
- **Risk score médio**. Separa fraude muito pouco (AUC 0,535). Em vez de um KPI, virou o ponto de comparação na curva.
- **Taxa por dia da semana e ranking de "piores dias do mês"**. Planos nesta base (de 2,32% a 2,74%). O mapa hora × dia já mostra essa ausência de padrão sem ocupar um KPI.
- **Valor total transacionado como número principal**. É indicador comercial, não de risco. Ficou como linha de contexto sob o KPI de transações.
- **Total de fraudes por tipo de transação**. A diferença entre compra, saque, transferência e pagamento é pequena (2,38% a 2,72%). Entraria como ruído.

4. Quem é o dono do painel?

- **Dono do negócio: o gerente de prevenção a fraudes**. Ele abre o painel toda segunda-feira, antes do comitê, e responde por duas perguntas: a regra de autenticação extra está alcançando as fraudes do recorte? O percentual de clientes legítimos recusados subiu? Quando a taxa de uma combinação que está fora da regra passa a ficar acima da taxa geral por semanas seguidas, é ele quem decide incluí-la.
- **Dono técnico: o analista de dados de risco**. Garante que o pipeline rodou (contagens do log batendo entre as camadas) e que a quarentena da Bronze está vazia ou tratada, antes de o painel ser atualizado.

Sem esses dois nomes definidos, o painel vira o "dashboard bonito que ninguém usa" discutido no dia 3 do curso.

Bônus — Modelo preditivo de fraude

Arquivos:

- `modelo_fraude.py`: treino e avaliação em PySpark local (Rota B), lendo a Silver da Etapa 2.
- `resultados/metricas_log.txt`: saída real da execução.
- `resultados/coeficientes.csv`: coeficientes, razões de chance, intervalos de confiança e p-valores.
- `resultados/captura_top.csv`: fraudes alcançadas nas transações mais suspeitas.
- `resultados/odds_ratios.png`: gráfico dos efeitos.

O que foi feito

Treinei uma Regressão Logística, o mesmo tipo de modelo do Lab 12, para estimar a probabilidade de cada transação ser fraude.

Variáveis de entrada. Todas estavam disponíveis no momento da transação:

- segmento, canal, categoria do estabelecimento e tipo de transação;
- se a transação foi de madrugada;
- risk score atual, credit score e valor.

Divisão dos dados. 80% para treino e 20% para teste, com semente fixa (42) para o resultado ser reproduzível. Ficaram 24.032 transações de treino (602 fraudes) e 5.968 de teste (149 fraudes). Todas as métricas abaixo são do teste, ou seja, de transações que o modelo não viu no treino.

Decisões e por quê

1. **O status ficou de fora.** Na Bronze (Etapa 2), constatei que 100% das fraudes estão como "declined". O status é, na prática, uma consequência do rótulo. Com ele, o modelo "acertaria" copiando a resposta, e a métrica pareceria excelente sem servir para nada: no momento de decidir se a transação é suspeita, o status ainda não existe.
2. **Categorias com referência explícita.** No Lab 12, o segmento virou um número único (0, 1, 2), o que obriga o modelo a supor que o risco cresce em degraus iguais. Aqui, cada categoria virou uma variável de sim ou não, comparada a uma referência escolhida: Premium, POS, varejo, compra e fora da madrugada. Assim cada coeficiente responde "quanto o risco muda em relação à referência". A madrugada segue a convenção da Silver (0h a 5h59).
3. **Escalas legíveis.** O risk score entra em blocos de 10 pontos e o credit score em blocos de 100. O valor entra em logaritmo, porque a distribuição é muito assimétrica.
4. **Sem regularização.** Deixei o modelo sem penalização para os coeficientes poderem ser interpretados diretamente. Para obter os p-valores, ajustei o mesmo modelo como GLM binomial no Spark; os coeficientes das duas versões são idênticos (diferença máxima de 0,0000).
5. **Métricas adequadas a evento raro.** Com 2,5% de fraude, um modelo que diz "nunca é fraude" acerta 97,5% das vezes. Por isso não usei acurácia. Usei:
 - a **AUC-ROC**, que mede se as fraudes recebem probabilidade maior que as legítimas;
 - a **AUC-PR**, mais exigente em eventos raros;
 - a **captura no topo**: quantas fraudes aparecem entre as X% transações com maior probabilidade.

Resultado

Métrica (teste)	Modelo	Só o risk_score atual	Sorteio
AUC-ROC	0,782	0,546	0,500
AUC-PR	0,096	0,031	0,025

Transações revisadas (maiores probabilidades)	Fraudes alcançadas pelo modelo	Fraudes alcançadas pelo risk_score
5% (298)	36, ou 24,2% (fraude no grupo: 12,08%)	12, ou 8,1% (4,03%)
10% (597)	58, ou 38,9% (9,72%)	29, ou 19,5% (4,86%)
20% (1.194)	82, ou 55,0% (6,87%)	40, ou 26,8% (3,35%)

Revisando 20% das transações, o modelo alcança o dobro das fraudes do score atual. Esse resultado, obtido fora da amostra de treino, é próximo ao da regra R6 da Etapa 3 (56,7% com 21,7% do volume). Isso confirma que o ganho das regras não foi um acaso da escolha feita sobre a própria base.

Interpretação dos coeficientes

A razão de chances (odds ratio) é o exponencial do coeficiente. Valor 1 significa sem efeito, 2 significa o dobro da chance de fraude em relação à referência. Todos os valores são do treino, com intervalo de confiança de 95%.

Efeitos fortes e claros ($p < 0,001$):

- **Segmento High-Risk: 11,4** (IC 9,1 a 14,2). Com canal, categoria e horário iguais, um cliente High-Risk tem cerca de 11 vezes a chance de fraude de um Premium. É o maior efeito do modelo, coerente com a análise 1.
- **Segmento Standard: 2,9** (IC 2,3 a 3,6) em relação ao Premium.
- **Canal app: 2,9** (IC 2,2 a 3,8) em relação ao POS. Web (1,37) e ATM (1,47) ficam pouco acima do POS, com p entre 0,04 e 0,05. O app é o único canal com efeito expressivo, como na análise 2.
- **Categoria viagem: 2,6** (IC 2,0 a 3,3) em relação ao varejo.
- **Madrugada: 1,6** (IC 1,3 a 1,9).

As outras categorias (saúde, alimentação, serviços, eletrônico) têm intervalos que cruzam o 1: não diferem do varejo. O mesmo vale para o tipo de transação (saque, transferência, pagamento).

Efeitos pequenos:

- **Risk score: 1,04 a cada 10 pontos** ($p = 0,013$). Estatisticamente existe, mas é mínimo: de 0 a 100 pontos, a chance aumenta cerca de 45%, enquanto o segmento sozinho multiplica por 11. A análise 5 mostra que esse efeito se concentra acima de 80 pontos, e uma reta não captura bem esse comportamento.
- **Credit score: 1,10 a cada 100 pontos** ($p = 0,004$). O sinal vai no sentido oposto ao intuitivo (score maior, um pouco mais de fraude) e o tamanho é pequeno. Com 17 variáveis testadas, é esperado que algum efeito pequeno apareça por acaso. A variável foi **mantida** no modelo: retirá-la só porque o resultado contraria a intuição seria ajustar o modelo ao resultado desejado. A conclusão prática, somada à análise 1, é não usar o credit score como critério de fraude.
- **Valor: 0,98 por unidade do logaritmo, ou 0,99 ao dobrar o valor** ($p = 0,64$). Sem efeito: fraudes e transações legítimas têm valores parecidos.

Leitura de negócio. O modelo confirma, com os efeitos separados uns dos outros, o que as análises descritivas mostraram: **quem** (segmento), **por onde** (app), **em quê** (viagem) e **quando** (madrugada). Esses quatro fatores explicam a fraude muito melhor que o score atual, o valor ou o credit score.

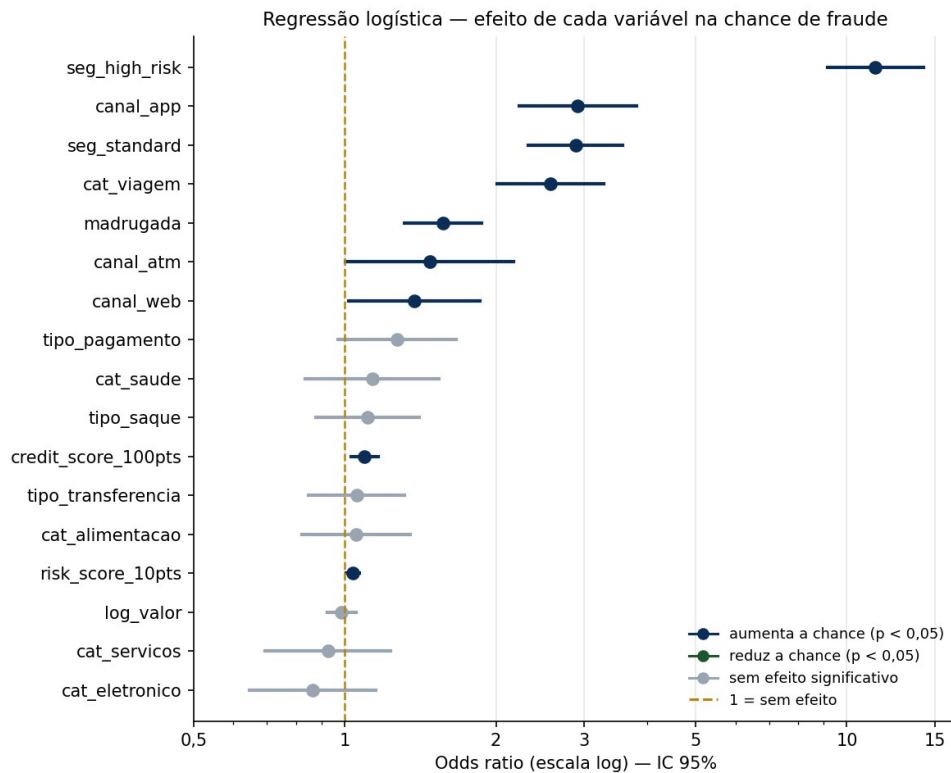


Figura 5 — Razão de chances de cada variável, com intervalo de confiança de 95% (escala logarítmica)

Limitações

- **Dados sintéticos.** Os efeitos recuperados ficam próximos dos multiplicadores do gerador do professor. O exercício valida o método, não a realidade de uma fintech.
- **Efeitos independentes.** O modelo assume que cada fator se soma aos outros. Interações como "app e viagem juntos" não foram incluídas como termo próprio.
- **Uma única divisão treino/teste.** Com 149 fraudes no teste, as métricas têm margem de erro relevante. Uma validação cruzada daria uma estimativa mais estável.
- **Probabilidades não calibradas para decisão.** O limite de corte deve ser escolhido pelo comitê pelo volume aceitável de transações revisadas, como na curva do dashboard, e não pelo limite padrão de 0,5: num evento de 2,5%, as probabilidades raramente passam desse valor, e praticamente nenhuma transação seria marcada.

Anexo — Estrutura do repositório e reprodução

O repositório reúne os 12 labs (pastas `dia1_fundamentos/`, `dia2_transformacao/` e `dia3_insights_bi/`) e a avaliação final (pasta `avaliacao_final/`). A tabela abaixo descreve os arquivos da avaliação.

Arquivo	Tipo	Função
<code>rodar_tudo.sh</code>	script	Executa toda a avaliação em sequência: base, diagrama, pipeline, teste, análises, dashboard e modelo
<code>etapa1_arquitetura/desenha_diagrama.py</code>	código	Desenha o diagrama (SVG e PNG)
<code>etapa1_arquitetura/justificativa.md</code>	relatório	Texto da Etapa 1
<code>etapa2_bigdata/generate_avaliacao_dataset.py</code>	código do professor	Gera a base de 30.000 transações (semente 123)
<code>etapa2_bigdata/pipeline.py</code>	código	Ingestão, raw, particionamento, Bronze, Silver e Gold
<code>etapa2_bigdata/teste_regras_bronze.py</code>	código	Injeta 12 linhas sujas numa cópia da base e confere o tratamento
<code>etapa2_bigdata/evidencias/</code>	saída	Logs reais da execução e do teste
<code>etapa2_bigdata/explicacao.md</code>	relatório	Texto da Etapa 2
<code>etapa3_analise/analises.py</code>	código	As cinco análises e a simulação de regras
<code>etapa3_analise/resultados/</code>	saída	Tabelas de cada análise
<code>etapa3_analise/dashboard.py</code>	código	Gera o painel a partir da Gold
<code>etapa3_analise/dashboard.html</code>	saída	Painel interativo
<code>etapa3_analise/achados.md</code> e <code>ideia_bi.md</code>	relatório	Textos da Etapa 3
<code>etapa4_ml/modelo_fraude.py</code>	código	Regressão logística em PySpark
<code>etapa4_ml/resultados/</code>	saída	Métricas, coeficientes e gráfico
<code>etapa4_ml/explicacao.md</code>	relatório	Texto do bônus

Reprodução. Com Python 3.10 ou superior e Java 17 ou superior instalados, basta instalar as bibliotecas listadas em `requirements.txt` e executar o script `rodar_tudo.sh` a partir da raiz do repositório. A execução completa leva poucos minutos e recria todas as camadas, tabelas, o dashboard e o modelo. Como a base é gerada com semente fixa, os números obtidos são idênticos aos deste relatório; essa reprodução foi verificada numa cópia limpa do repositório.

Arquivos não versionados. Os CSVs e os Parquets das camadas não vão para o Git (ver `.gitignore`), porque são recriados pelos scripts. As exceções são as tabelas pequenas de resultado das análises e do modelo.